COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	07-09-2026	**DATE SUBMITTED:**	07-09-2026
NAME:	DOLOSO JR, ANDREW S.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. HOA1.1 [BASIC C++ PROGRAMMING]

1. PROCEDURE OUTPUT

To implement the stack ADT in C++
To create an implementation of stack with different internal representations

ILO_A

```
#include<iostream>
#include<stack>

int main(){

    std::stack<int> newStack;

newStack.push(3);
newStack.push(8);
newStack.push(15);

std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
std::cout<<"Removing elements from the stack: \n";
newStack.pop();
std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
newStack.pop();
std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
newStack.pop();
std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

return 0;
}
```

ILO_A OUTPUT

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
PS C:\Users\TIPQC\Desktop\CPE21S1_DOLOSO> & 'c:\Users\TIPQC\
soft-MIEngine-Error-vb5v0410.533' '--pid=Microsoft-MIEngine-P
Stack Empty? 0
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is: 8
The top of the element is: 3
Stack Empty? 1
PS C:\Users\TIPQC\Desktop\CPE21S1_DOLOSO>
```

ILO_B 1

```cpp
#include <iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;
//always -1 is the top of the stack if it is empty

//functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();

int main() {
   int choice;
   std::cout << "Enter the number of max elements for the new stack: ";
   std::cin >> i;
```

```
    push();

    while(true) {
     std::cout << "stack operations:" << std::endl;
     std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty" << std::endl;
     std::cin >> choice;
         switch(choice) {
         case 1: push();
         break;
         case 2: pop();
         break;
         case 3: Top();
         break;
         case 4: std::cout << isEmpty() << std::endl;
         break;
         default: std::cout << "invalid choice." << std::endl;
         break;
        }
    }
    return 0;
}

bool isEmpty() {
    //create a condition to check whether the top element is equal to -1
    if (top == -1) return true;
    return false;
}

bool isFull() {
    //check if we reached the maximum capacity of the array
    if (top >= i-1) return true;
    return false;
}

void push() {
    //check if full -> if yes return an error
    if (isFull()) {
        std::cout << "stack overflow" << std::endl;
```

```
        return;
    }

    std::cout << "New Value to add into the stack" << std::endl;
    std::cin >> newData;
    //increment the top(indext by 1) then assign the new data;
    stack[++top] = newData;
}

void Top() {
    //check if the stack is empty
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
    }
    std::cout << "The element on the top of the stack is " << stack[top] <<
std::endl;
}

void pop() {
    //check if the stack is empty
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
    }
    //display the top value
    std::cout << "popping" << stack[top] << std::endl;
    top--;
}
```

```
Enter the number of max elements for the new stack: 1
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
2
stack underflow
popping0
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
3
The element on the top of the stack is 0
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
4
0
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
3
The element on the top of the stack is 0
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
2
popping0
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
1
New Value to add into the stack
4
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
1
New Value to add into the stack
2
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
3
stack underflow
The element on the top of the stack is 2
Stack Operations:
 1. PUSH 2.POP 3.TOP 4.isEmpty
```

ILO_B 2

```cpp
#include <iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;
//always -1 is the top of the stack if it is empty
```

```
//functions
void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void displayStack();

int main(){
   int choice;
   std::cout << "Enter the number of max elements for the new stack: ";
   std::cin >> i;

   push();

   while(true) {
    std::cout << "stack operations:" << std::endl;
    std::cout << "1. PUSH 2. POP 3. TOP 4. isEmpty 5. Displa Stack" <<
std::endl;
    std::cin >> choice;
        switch(choice) {
        case 1: push();
        break;
        case 2: pop();
        break;
        case 3: Top();
        break;
        case 4: std::cout<<isEmpty()<<std::endl;
        break;
        case 5: displayStack();
        break;
        default: std::cout << "invalid choice." << std::endl;
        break;
       }
   }
   return 0;
}
```

```
bool isEmpty() {
   //create a condition to check whether the top element is equal to -1
   if (top == -1) return true;
   return false;
}

bool isFull() {
   //check if we reached the maximum capacity of the array
   if (top >= i-1) return true;
   return false;
}

void push() {
   //check if full -> if yes return an error
   if (isFull()) {
       std::cout << "stack overflow" << std::endl;
       return;
   }
   std::cout << "New Value to add into the stack" << std::endl;
   std::cin >> newData;
   //increment the top(indext by 1) then assign the new data;
   stack[++top] = newData;
}

void Top() {
   //check if the stack is empty
   if (isEmpty()) {
       std::cout << "stack underflow" << std::endl;
   }
   std::cout << "The element on the top of the stack is " << stack[top] <<
std::endl;
}

void pop() {
   //check if the stack is empty
   if (isEmpty()) {
       std::cout << "stack underflow" << std::endl;
```

```
    }
    //display the top value
    std::cout << "popping" << stack[top] << std::endl;
    top--;
}
void displayStack(){
// condition to check if the stack is empty
if(isEmpty()){
    std::cout<<"Cannot print an Empty stack"<<std::endl;
}
//we have create a loop that iterates the top element
for(int i = top; i >= 0; i --){
    std::cout<<stack[i]<< ',';
    }
    //add a new line
    std::cout<<std::endl;

}
```

```
PS C:\Users\TIPQC\Desktop\CPE21S1_DOLOSO>  & 'c:\Users\TIPQC\.vscod
gAdapters\bin\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine
d4x0l5.o0s' '--stderr=Microsoft-MIEngine-Error-uedqwop4.dvx' '--pid
ys64\ucrt64\bin\gdb.exe' '--interpreter=mi'
Enter the number of max elements for the new stack: 2
New Value to add into the stack
1
stack operations:
1. PUSH 2. POP 3. TOP 4. isEmpty 5. Displa Stack
1
New Value to add into the stack
2
stack operations:
1. PUSH 2. POP 3. TOP 4. isEmpty 5. Displa Stack
5
2,1,
stack operations:
1. PUSH 2. POP 3. TOP 4. isEmpty 5. Displa Stack
```

DISCUSSION / CODE ANALYSIS

In this activity, we learned how to use the stacks and implements using a static array and demonstrates the basic stack operations: push, pop, top, isEmpty, isFull, and display. It follows the Last In, First Out (LIFO) principle, where the last element added is the first one removed. The program also checks for stack overflow and

underflow to prevent invalid operations. Overall, it provides a simple and efficient way to understand how a stack data structure works in C++.

ILO_C

MAIN.CPP

```
#include <iostream>
#include "stackList.h"


int main() {


    std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---- \n";


    // push
    push<char>('C');
    push<char>('P');
    push<char>('E');


    // Display the current top
    top<char>(); // Top should be 'E'


    std::cout << "\n ----- Display all the elements ----- \n";
    displayAll<char>();

    std::cout << "\n ----- Popping all the elements ----- \n";


    while (!isEmpty<char>()) {
        std::cout << " Popped: " << pop<char>() << std::endl;
    }


    std::cout << "\n ----- Testing the Underflow ----- \n";
```

```
    pop<char>();


    return 0;
}
```

STACKLIST.H

```
#ifndef STACK_LIST_H
#define STACK_LIST_H


// class node that contains data and the next
template <typename T>
class Node {
public:
    T data;       // VALUE THAT WE NEED STORE
    Node* next;   // POINTER TO THE NEXT NODE
};


template <typename T>
Node<T>* head = nullptr;


// isEmpty function (placed before pop/top/display to avoid forward
declaration errors)
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}


// push
template <typename T>
void push(T newData) {
    // create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
```

```
    // new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}


// pop
template <typename T>
T pop() {
    // checks if the stack is empty
    if (head<T> == nullptr) {
        std::cout<<"Stack underflow." << std::endl;
        return T();
    }


    // save the current top node
    Node<T>* temp = head<T>;
    // store its data
    T tempVal = temp->data;


    // move the head to the next node
    head<T> = head<T>->next;
    // free the removed node
    delete temp;
    // return the removed value
    return tempVal;
}


// top
template <typename T>
void top() {
    // error catching:
    if (isEmpty<T>()) {
```

```
        std::cout << "Stack underflow" << std::endl;
        return;
    }


    // display the Top Node
    std::cout << "Top of stack: " << head<T>->data << std::endl;
}


// Display all
template <typename T>
void displayAll() {
    if (isEmpty<T>()) {
        std::cout << "Stack is empty" << std::endl;
        return;
    }


    std::cout << "Stack contents (top to bottom): ";
    Node<T>* temp = head<T>;
    while (temp != nullptr) {
        std::cout << "\n| " << temp->data << " | ";
        temp = temp->next;
    }
    std::cout << std::endl;
}

#endif
```

```
[Running] cd "c:\Users\Doloso\CPE010_DOLOSO\HOA4P1\ILOC\" && g++ main.cpp -o main &&
"c:\Users\Doloso\CPE010_DOLOSO\HOA4P1\ILOC\"main
--- PUSHING 'C' 'P' 'E' ONTO THE STACK ----
Top of stack: E

----- Display all the elements -----
Stack contents (top to bottom):
| E |
| P |
| C |

----- Popping all the elements -----
Popped: E
Popped: P
Popped: C

----- Testing the Underflow -----
Stack underflow.
```

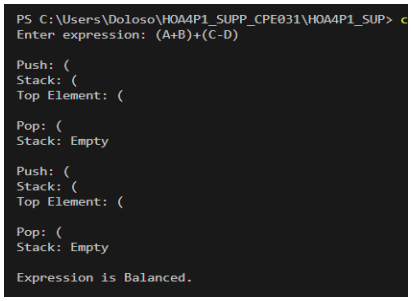
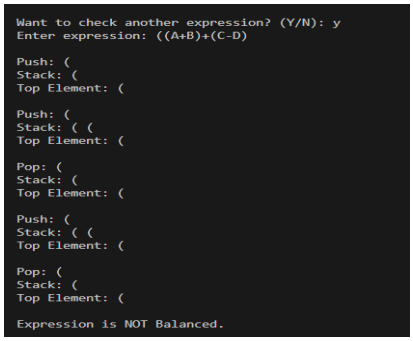
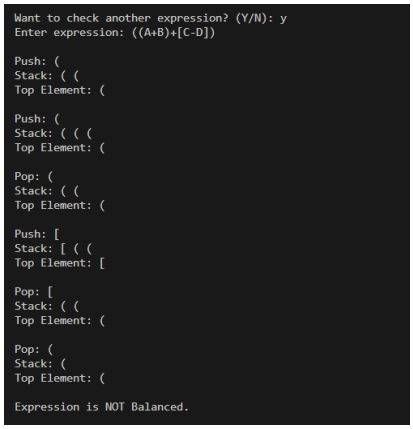
ANALYSIS:

This activity helped me understand how to implement a stack using a linked list and apply the Last In, First Out (LIFO) principle through the push, pop, top, and isEmpty functions. I also learned how templates make the program reusable for different data types, while pointers and dynamic memory allocation allow the stack to grow and shrink efficiently. Overall, this activity improved my understanding of stack operations and linked list implementation in C++.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

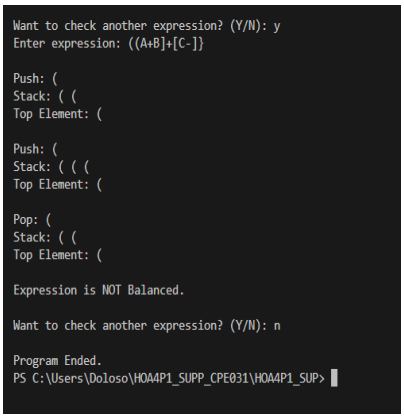
ILO C: Solve problems using an implementation of stack:
The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL
Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {,or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.
Steps:
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:

i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.
Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		has a matching closing parenthesis). Both pairs (A+B) and (C-D) are properly closed, leaving the stack empty at the end.
((A+B)+(C-D)	N		One extra opening parenthesis remains in the stack at the end, making the expression unbalanced.
((A+B)+[C-D])	N		There are three opening symbols ((, (, and [) and three closing symbols (),], and)). However, an extra outer parenthesis (at the very beginning is never matched by a final closing parenthesis, leaving (remaining in the stack.

COMPUTER ENGINEERING

QUEZON CITY

((A+B]+[C-D]}	N		The second opening parenthesis (encounters a closing square bracket] instead of a), causing the stack check to fail/halt early due to symbol mismatch and improper opening/closing sequences.

CODE

```
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

void push(char data);
char pop();
void Top();
bool isEmpty();
bool isFull();
bool isBalanced(std::string expression);
void displayStack();

int main() {

    std::string expression;
    char choice;

    do {
        std::cout << "Enter expression: ";
        std::getline(std::cin, expression);
```

```
    if (isBalanced(expression))
        std::cout << "\nExpression is Balanced." << std::endl;
    else
        std::cout << "\nExpression is NOT Balanced." << std::endl;
        std::cout << "\nWant to check another expression? (Y/N): ";
        std::cin >> choice;
        std::cin.ignore();
    } while (choice == 'Y' || choice == 'y');

        std::cout << "\nProgram Ended." << std::endl;

    return 0;
}

// Check if stack is empty
bool isEmpty() {
    return top == -1;
}

// Check if stack is full
bool isFull() {
    return top == maxCap - 1;
}

// Push
void push(char data) {
    if (isFull()) {
        std::cout << "Stack Overflow!" << std::endl;
        return;
    }

    stack[++top] = data;

        std::cout << "\nPush: " << data << std::endl;
    displayStack();
}
```

```
// Pop
char pop() {
    if (isEmpty()) {
        return '\0';
    }

    char temp = stack[top--];

    std::cout << "\nPop: " << temp << std::endl;
    displayStack();

    return temp;
}

// Display Top
void Top() {
    if (isEmpty()) {
        std::cout << "Top: Stack is Empty" << std::endl;
        return;
    }

    std::cout << "Top Element: " << stack[top] << std::endl;
}

// Display Stack
void displayStack() {

    if (isEmpty()) {
        std::cout << "Stack: Empty" << std::endl;
        return;
    }

    std::cout << "Stack: ";

    for (int i = top; i >= 0; i--) {
        std::cout << stack[i] << " ";
    }
```

```
    std::cout << std::endl;

    Top();
}

// Balanced Symbols
bool isBalanced(std::string expression) {
    for (int i = 0; i < expression.length(); i++) {

        char ch = expression[i];

    // Opening Symbols
    if (ch == '(' || ch == '[' || ch == '{') {
        push(ch);
    }

    // Closing Symbols
    else if (ch == ')' || ch == ']' || ch == '}') {

        if (isEmpty()) {

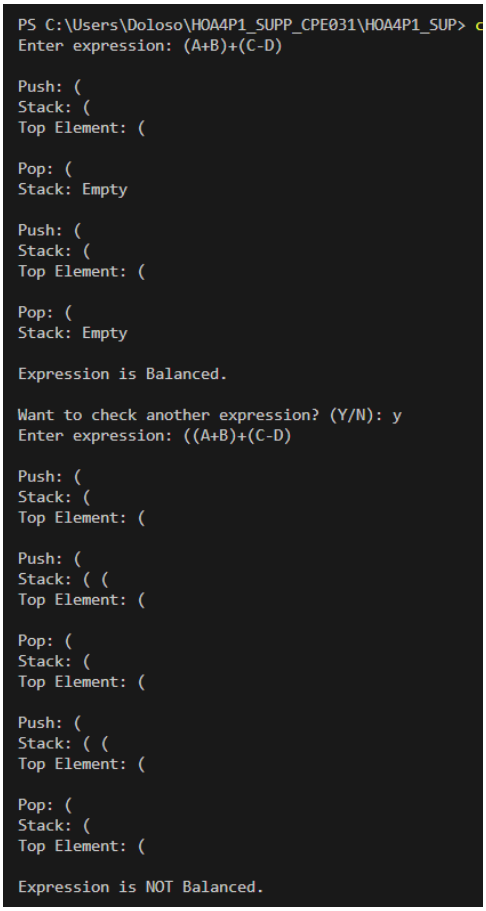
    return false;
    }

    char open = pop();

    if ((ch == ')' && open != '(') ||
        (ch == ']' && open != '[') ||
        (ch == '}' && open != '{')) {
        return false;
    }
    }
    }

    return isEmpty();
}
```

OUTPUT



```
PS C:\Users\Doloso\HOA4P1_SUPP_CPE031\HOA4P1_SUP> c
Enter expression: (A+B)+(C-D)

Push: (
Stack: (
Top Element: (

Pop: (
Stack: Empty

Push: (
Stack: (
Top Element: (

Pop: (
Stack: Empty

Expression is Balanced.

Want to check another expression? (Y/N): y
Enter expression: ((A+B)+(C-D)

Push: (
Stack: (
Top Element: (

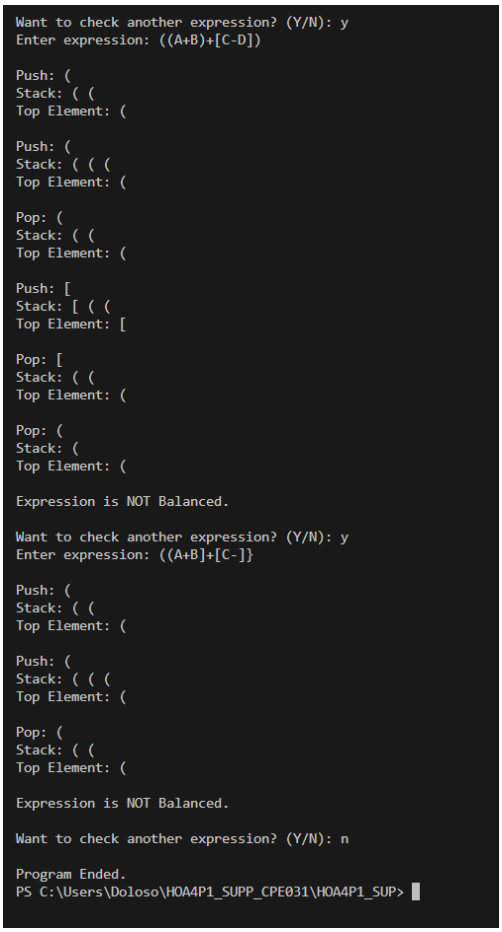
Push: (
Stack: ( (
Top Element: (

Pop: (
Stack: (
Top Element: (

Push: (
Stack: ( (
Top Element: (

Pop: (
Stack: (
Top Element: (

Expression is NOT Balanced.
```



```
Want to check another expression? (Y/N): y
Enter expression: ((A+B)+[C-D])

Push: (
Stack: ( (
Top Element: (

Push: (
Stack: ( ( (
Top Element: (

Pop: (
Stack: ( (
Top Element: (

Push: [
Stack: [ ( (
Top Element: [

Pop: [
Stack: ( (
Top Element: (

Pop: (
Stack: (
Top Element: (

Expression is NOT Balanced.

Want to check another expression? (Y/N): y
Enter expression: ((A+B]+[C-]}

Push: (
Stack: ( (
Top Element: (

Push: (
Stack: ( ( (
Top Element: (

Pop: (
Stack: ( (
Top Element: (

Expression is NOT Balanced.

Want to check another expression? (Y/N): n

Program Ended.
PS C:\Users\Doloso\HOA4P1_SUPP_CPE031\HOA4P1_SUP>
```

Tools Analysis
How do the different internal representations affect the implementation and usage of the stack?

In this activity, I learned that the implementation of a stack affects the way it stores and manages data, even though both implementations produce the same result. The array implementation is easier to understand but has a fixed size, while the linked list implementation is more flexible because it can grow as needed. This activity helped me understand how stacks are used to check if symbols in an expression are balanced.

Conclusion / Learning

Summary of lessons learned:

In this activity, I learned how stacks work using both arrays and linked lists.I also learned the purpose of push, pop, top and display operations are used to solve real problems like checking whether an expression has balanced symbols. This activity helped me understand the importance of the LIFO principle and its importance in stack operations.

Analysis of the procedure:
The implementation of steps on this program were simple, but when applying the code you had to be careful with the syntax and the logic. I made some mistakes while coding like typo's, unmatched braces, wrong function declarations, but correcting them helped me to learn the program better.

Analysis of the supplementary activity:

This supplementary activity allowed me to use what I had learned about stacks in a real-life scenario. By testing different expressions and observe how the stack stores opening symbols and removes them when matching closing symbols are encountered. This helped me better understand the role of stacks in checking balanced expressions.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

I believe I still need more practice and exercises to improve my programming skills because I was not able to complete the program independently. After receiving guidance from my instructor and classmates in correcting several coding errors, I realized that I need to continue practicing debugging and writing programs on my own. Despite these challenges, this activity increased my knowledge of stacks and helped me better understand their applications in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: *Grammarly*

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: *grammar correction*

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.